Ingenieria de software 1

Jaime Nazar Anchorena

Contents

Introduccion **2**

Unidad 1: Ciclos de vida **2**

Que es 2

Por que 2

Modelos de ciclos de vida 2

Cascada 2

Incremental 4

Evolutivo 4

Prototipado 4

Espiral 6

Proceso unificado 6

Esfuerzo 6

Mejores practicas 6

Errores comunes 10

Introduccion

Nosotros no buscamos la solucion optima teorica, sino buscamos que podemos hacer con nuestras limitacion de tiempo y economica. La ingenieria de software tiene dos dimensiones importantes, tiempo y escala(cuantas personas necesitamos para desarrollar una solucion y como se van a mantener los equipos).

Definicion interesante(suena mejor en ingles): Desarrollo entre multiples personas de programas de multiples definiciones. *Multi-person development of multi-version software.* Es decir, debe ir evolucionando, nunca se queda como esta ni se descarta y se empieza de nuevo.

Unidad 1: Ciclos de vida

Que es

Proceso sistematico y estandarizado por el cual un producto de software es desarrollado y mantenido a lo largo del tiempo.

Define un marco común para los procesos del ciclo de vida de los programas informáticos, con una terminología bien definida, a la que pueda remitirse la industria del software. Contiene procesos, actividades y tareas aplicables durante la adquisición, el suministro, el desarrollo, el funcionamiento, el mantenimiento o la eliminación de sistemas, productos y servicios informáticos. Estos procesos del ciclo de vida se llevan a cabo mediante la participación de los interesados, con el objetivo final de lograr la satisfacción del cliente".

Por que

Sirve para contruir **software de alta calidad**, ese el el unico motivo. Esto tambien implica:

- **Predecir la etapa del ciclo de desarrollo**: Licencias, oficinas, personal, equipos, telefonos etc. Todo lo que necesitemos.
- Minimizar errores
- Minimizar costos
- **Reutilizable y facilmente mantenible**: Al aplicar determinados mecanismos y tecnologias vamos a poder hacer el software mas reutilizable y mantenible, entonces, menos costoso.
- Cumplir con los rquisitos del cliente
- Escalabilidad

Modelos de ciclos de vida

Todos los modelos tienen sus pros y sus contras, de manera que dependera de la naturaleza del projecto cual sera el modelo a utilizar.

Cascada

La finalizacion de una etapa implica en inicio de la siguiente. Las etapas son:

1. Analysis
2. Requirements specification
3. Design
4. Development

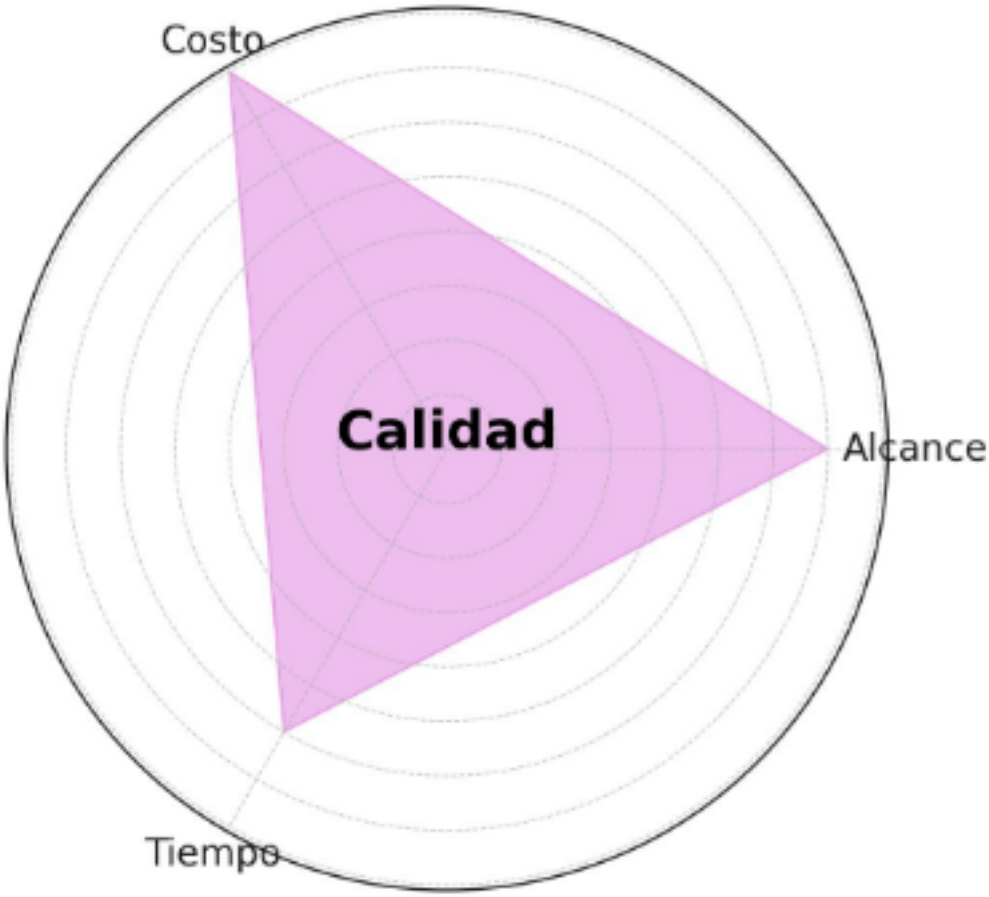

Figure 1: Representacion de la calidad de un software

5. Testing and integration
6. Implementation/Deployment

Si me doy cuenta que, por ejemplo, durante la etapa de **testing** nos damos cuenta que hay que introducir un nuevo requerimiento, va a haber que volver a empezar(volver al analisis).

Ademas, lo primero de software que vera el cliente sera en la etapa final, lo cual puede ser meses o anos despues. Esto introduce el problema de que el cliente puede no estar satisfecho con el resultado final.

Incremental

Plantea resolver el problema principal del modelo cascada, trata de mostrar software funcionando lo antes posible.

Primero se define el sitema copleto y se lo divide en funcionalidades, luego, cada incrmento sigue todas las fases del ciclo de vida (análisis -> diseño -> codificación -> pruebas -> entrega).

Se entrega al usuario **funcionalidad completa por partes**.

Por ejemplo: Sobre Gmail: Google Meet, Google Classroom, Google Jamboard.

Evolutivo

Busca recoger **feedback** rapidamente, para poder mejorar la implementacion. Con cada ciclo se entregan diferentes versiones. Como funciona:

- Se define un conjunto minimo de requerimientos
- Se desarrolla una primera version o prototipo
- Se entrega al usuario y se recoge feedback
- Se planifican mejoras y se desarrolla una nueva version
- El ciclo se repite hasta alcanzar el producto final

Hay dos tipos de prototipos, el desechable(Tener cuidado con las expectativas del cliente, puede ser que no le guste que este *atado con alambre*) y el que es simplemente una version del programa, en la cual se puede ir iterando.

Modelo de desarrollo que , a diferencia del de Prototipos, busca reemplazar el viejo sistema con uno nuevo que tendría la propiedad de satisfacer los nuevos requerimientos lo más rápido posible. El desarrollo evolutivo asume que los requerimientos están sujetos a cambios continuos y que la estrategia para enfrentar aquello pasa por un reflejo, también continuo de aquellos cambios.

Por ejemplo: Windows, Office, Ubuntu.

> **Nota**: La pricipal falla de los proyectos informaticos es la falta de **comunicacion**. Esto es reponsabilidad del Ingeniero de Software, el prototipado busca resolver parte de esta comunicacion.

Prototipado

La idea es poder mostrar como pensas que puede ser el sistema. Modelo que pone énfasis en la etapa de Especificación de Requerimientos a través de la construcción de prototipos que aproximan al usuario a la idea final del sistema, con objeto de poder clarificar los requerimientos.

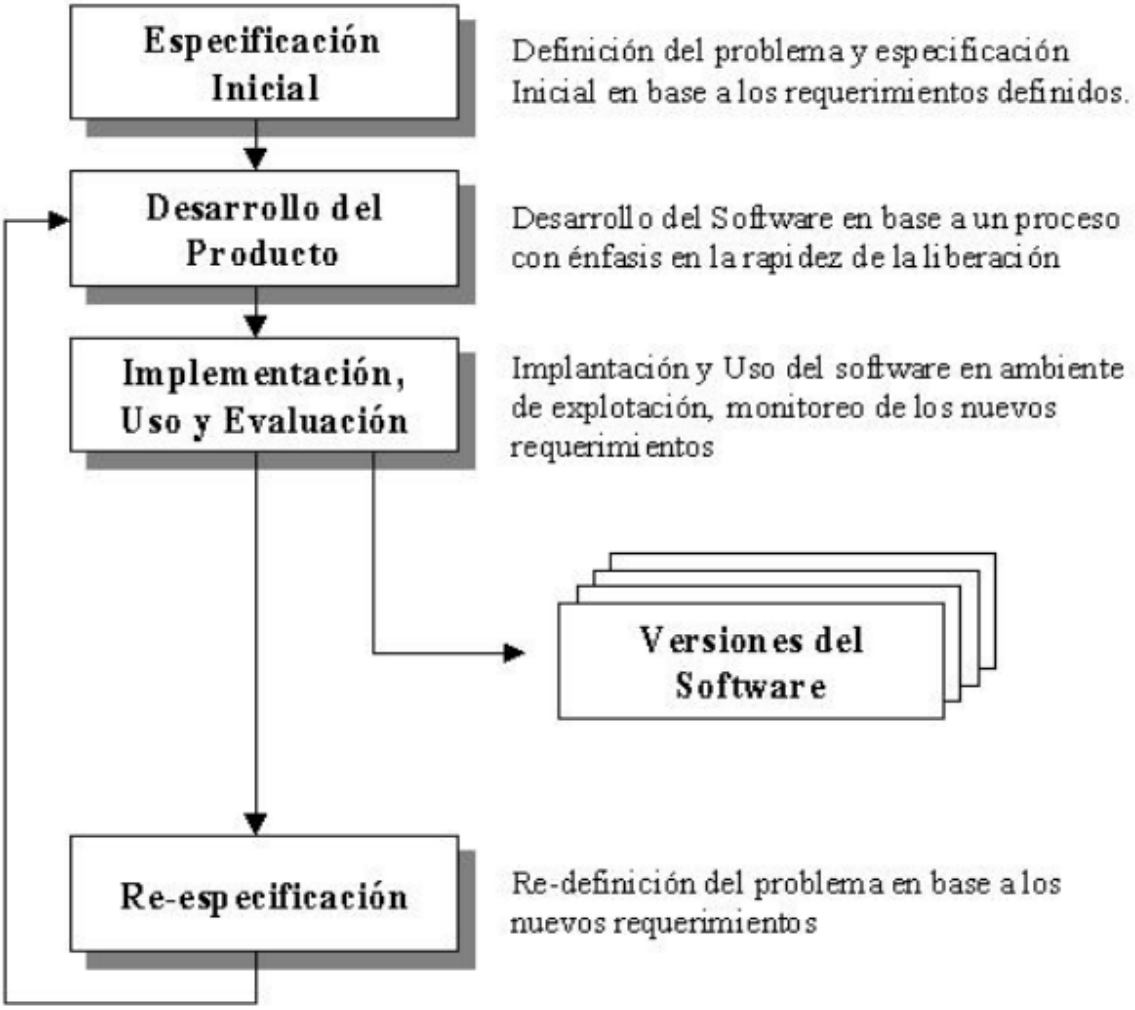

Figure 2: Modelo evolutivo

Sirve para poder descubrir la funcinalidad completa que quiere el cliente. Esto se debe a que el cliente generalmente no sabe muy bien lo que quiere, hay que validad lo mas posible si estamos desarrollando lo que quiere el cliente.

Espiral

El mas completo, usa todos los anteriores:

- Prototipado
- Evolutivo
- Iterativo
- Incremental

Una cosa muy importante que contempla este modelo es el analisis de riesgos, ademas de su gestion dentro de cada iteracion. Trata las falencias de los modelos anteriores.

Proceso unificado

De una manera es la madre de todos estos projectos. UP es un framework de proceso detallado, define aproximadamente 50 opcionales artefactors(non-software work products), tales como:

- Vision, Lista de riesgos, etc
- En RUP estan organizados dentro de las diciplinas, y los encontramos como templates(la documentacion)
- La regla a seguir con respecto a la creacion de estos documentos es "Menos es mejor"
- Esta unica adaptacion es llamada **Development Case** del projecto

Esto esta basado en projectos anteriores, tomando las practicas que funcionaron. No hace falta usar todos los artefactos en nuestros projectos. No quieren que agamos documentacion por documentacion, sino que tenga un *target*.

Esfuerzo

Los recursos y el staff son variables.

Mejores practicas

- **Iteraciones TimeBoxed**: Recomendadas entre 2 y 6 semanas, como hackatlon.
- **Programando los elementos de alto riesgo, y alto valor**: Incentivar a la reutilización de componentes
- **Verificación de calidad continuada**: Uso de técnicas de integración y testeo continuo, como XP
- **Modelado visual**: Esquemas en una pizarra durante 1 hora. Esquemas basados en UML
- **Administración de requerimientos**: Encontrar (req. Funcionales a través de workshops). Organizar (según prioridades, atributos, riesgos, ...) y seguir los requerimientos (según su estado – asignado, en curso, ...)
- **Administración del cambio**: A través de la administración de la config. y del control de versión. El uso de un protocolo de solicitud de cambios. Releases al final de cada iteración

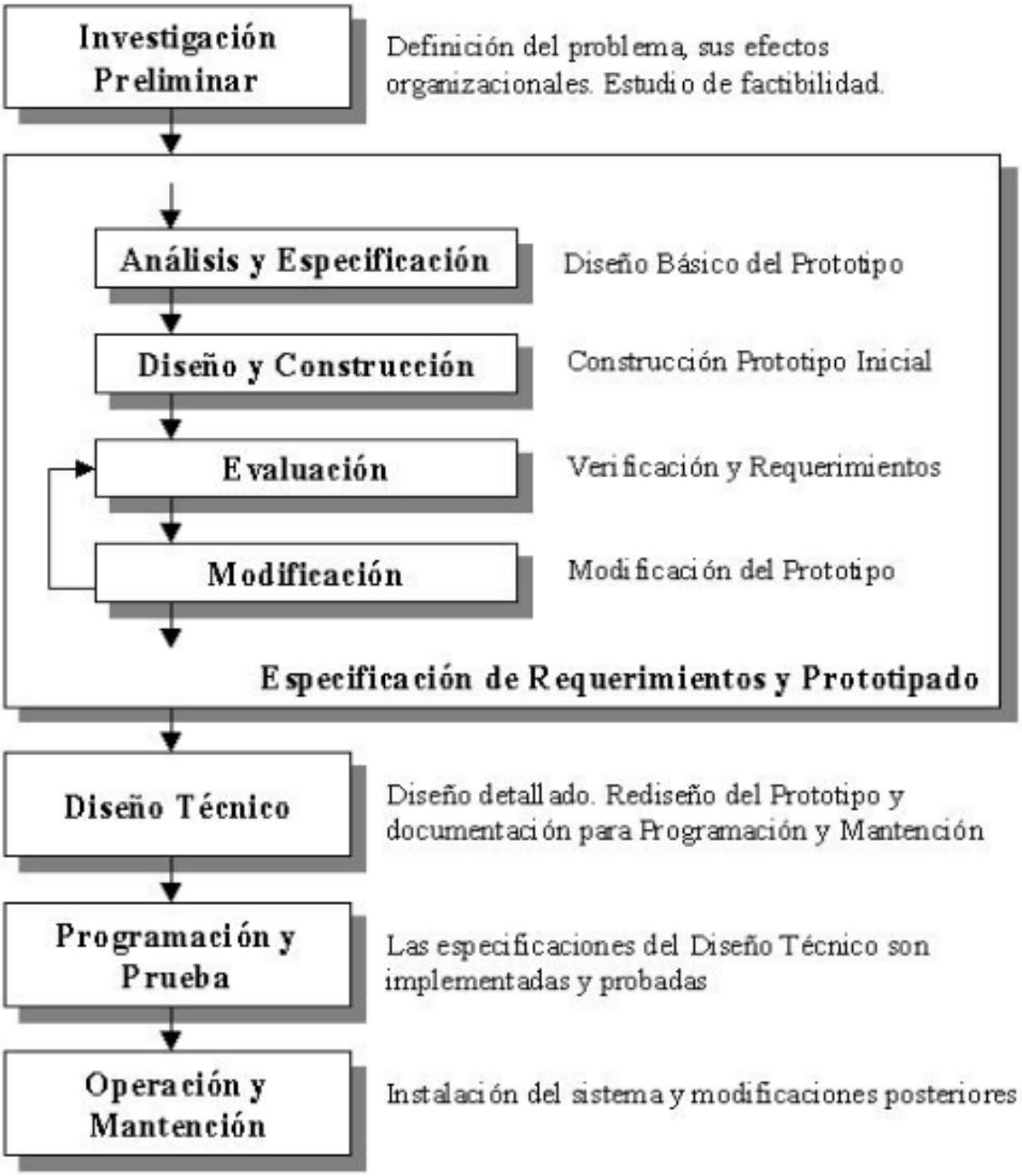

Figure 3: Modelo prototipado

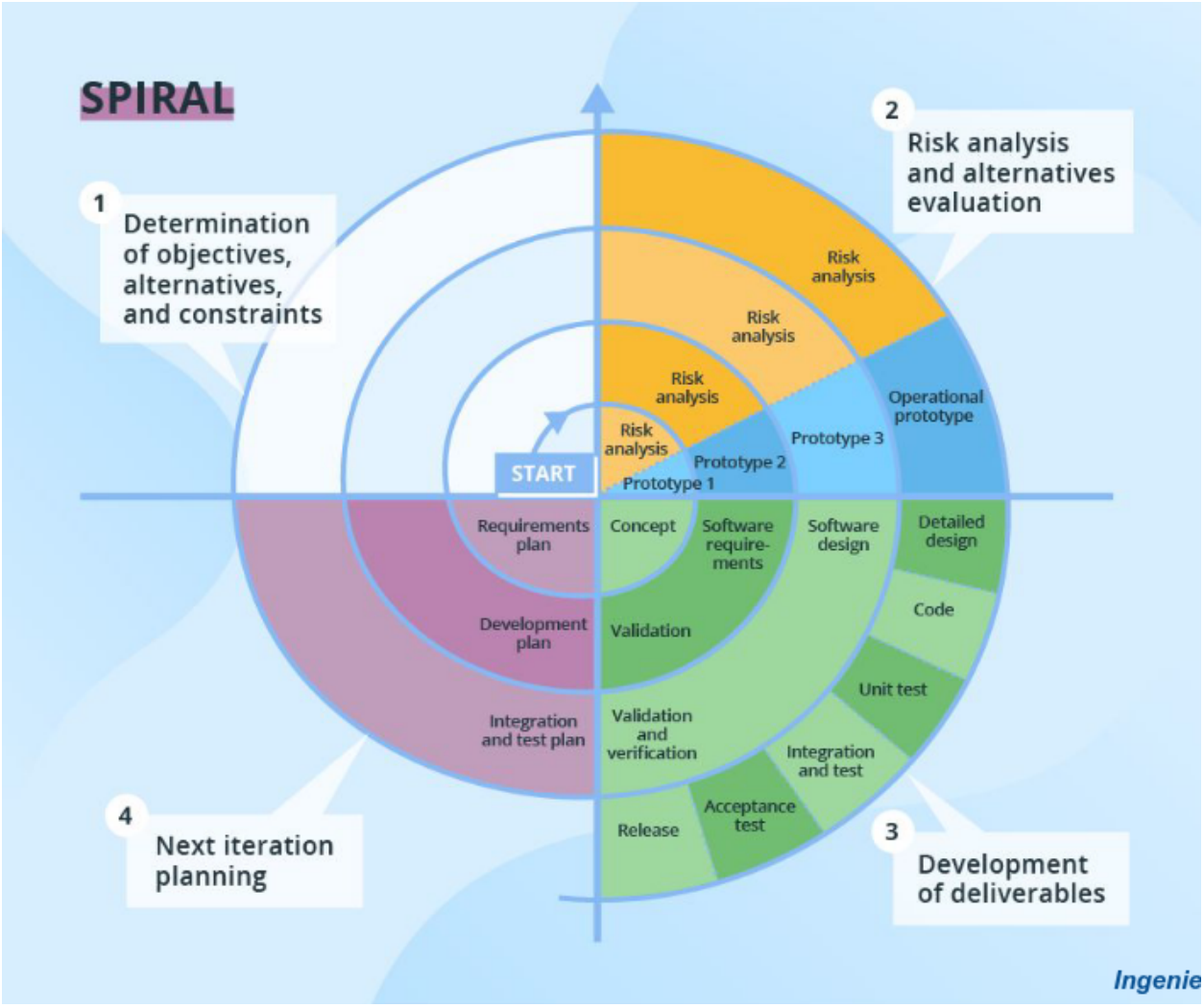

Figure 4: Modelo espiral

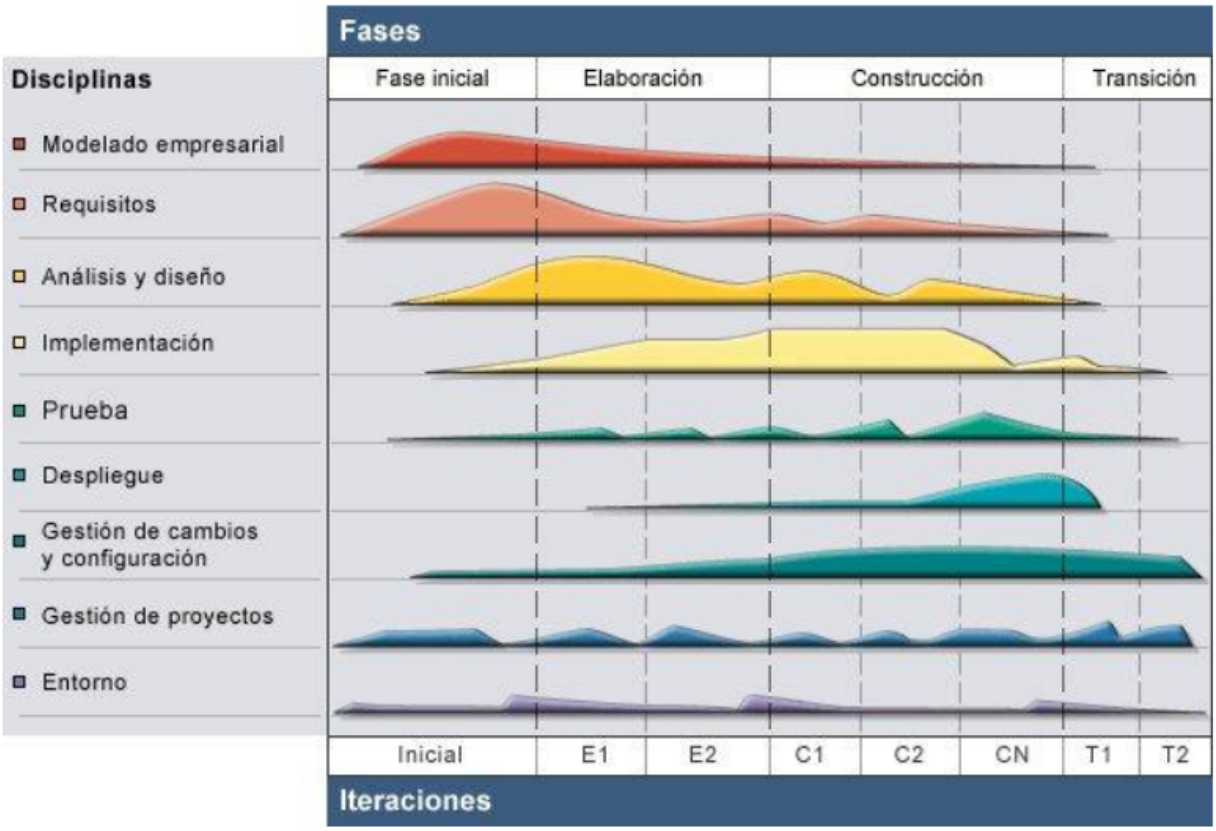

Figure 5: Actividad por diciplinas RUP

Errores comunes

- **Iteraciones**: Demasiado largas. Cada iteración no finaliza con un release en producción
- **El objetivo de la fase elaboración**: Crear un prototipo desechable
- **Planeamiento**: Predictivo no adaptativo. Diferir el testeo mayor hasta cerca de finalizar el proyecto
- **Documentación**: Hacer cantidades de diagramas de UML. Realizar un "Development Case" demasiado complejo, con muchos work products. El Software Architecture Document (SAD) finalizado antes de terminar la elaboración